

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Bheeshma L (192511332) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. Suppose a software system is developed for a **Library Management System** where users can search books, borrow, and return them.

(a) Explain the fundamental object-oriented concepts such as **class, object, encapsulation, inheritance, and polymorphism** in this context.

- i. Class: Blueprint (e.g., Book, User)
- ii. Object: Instance of class
- iii. Encapsulation: Data hiding using methods
- iv. Inheritance: Reusability (e.g., Student inherits User)
- v. Polymorphism: Same method, different behavior

Answer: In the Library Management System, a Class such as Book or User acts as a blueprint that defines the common attributes and methods for all books or users. An Object is a specific instance of a class, for example a particular book with its own title, author and ISBN.

Encapsulation is achieved by keeping the internal data of a class private and exposing it only through public methods, such as borrowBook() or returnBook(), which protects the data from unauthorized access. Inheritance allows a new class, such as Student, to reuse the properties and methods of an existing class, such as User, avoiding duplicate code. Polymorphism allows the same method, such as displayDetails(), to behave differently depending on whether it is called on a Book object or a User object.

(b) Describe different types of **object relationships** such as association, aggregation, and composition.

- i. Association: General connection
- ii. Aggregation: Weak “has-a” relationship
- iii. Composition: Strong ownership

Answer: Association represents a general relationship between two independent classes, such as a User borrowing a Book, where both classes can exist without each other. Aggregation is a weaker form of the “has-a” relationship in which one class contains another, but the contained object can exist independently, such as a Library having Books that could also exist outside the library catalogue. Composition is a stronger form of ownership where the contained object cannot exist without the owner, for example a Library System containing a Catalogue module that has no meaning outside the system.

(c) Illustrate how these concepts help in building a modular and reusable system.

- Improves maintainability, flexibility, and scalability

Answer: These object-oriented concepts help build a modular and reusable system because classes can be developed, tested and modified independently of one another. Encapsulation protects data integrity, inheritance promotes code reuse by allowing new classes to extend existing ones, and polymorphism allows a single interface to work with different object types. Together they make the Library Management System easier to maintain, extend with new features, and scale as the number of users and books grows.

2. Consider developing a **Student Information System**.

(a) Explain key object-oriented design principles such as:

- i. **Abstraction** – Showing essential features
- ii. **Encapsulation** – Protecting data
- iii. **Modularity** – Dividing into smaller components

Answer: Abstraction focuses on showing only the essential features of an entity, such as a Student's name, roll number and courses, while hiding the internal implementation details from the user. Encapsulation protects the internal data of the Student class by restricting direct access and allowing changes only through defined methods, which prevents accidental data corruption. Modularity divides the Student Information System into smaller, independent components such as attendance, marks and fee management modules, making the system easier to develop, test and maintain.

(b) Describe principles like **Single Responsibility Principle (SRP)** and **Open/Closed Principle (OCP)**.

- i. SRP: One class → one responsibility
- ii. OCP: Open for extension, closed for modification

Answer: The Single Responsibility Principle states that a class should have only one reason to change, meaning each class should be responsible for a single functionality, such as a StudentRecord class handling only student data and not fee processing. The Open/Closed Principle states that classes should be open for extension but closed for modification, so new features can be added through inheritance or interfaces without altering existing, tested code.

(c) Explain how these principles improve software quality.

- Reduces complexity, improves code reuse and maintainability

Answer: Applying these principles improves software quality by reducing the complexity of the Student Information System, since each class has a clear and limited responsibility. It also improves code reusability, since well-defined modules can be reused in other parts of the system, and makes the system easier to maintain because changes to one module have minimal impact on others.

3. Suppose a company plans to develop an **Online Shopping System**.

(a) Explain the phases of SDLC:

- i. Requirement Analysis
- ii. Design
- iii. Implementation
- iv. Testing
- v. Deployment
- vi. Maintenance

Answer: The Software Development Life Cycle for the Online Shopping System begins with Requirement Analysis, where the needs of customers and administrators are gathered. This is followed by Design, where the system architecture and database structure are planned, and Implementation, where the actual coding of modules such as the product catalogue and

payment gateway takes place. Testing verifies that the system works correctly and is free of defects, Deployment makes the system available to end users, and Maintenance involves fixing issues and adding improvements after release.

(b) Describe how each phase contributes to structured development.

- Ensures proper planning and execution

Answer: Each phase of the SDLC contributes to structured development by ensuring that work proceeds in a logical and organised sequence. Requirement analysis and design lay a solid foundation before any code is written, implementation and testing ensure the product meets its specifications, and deployment and maintenance ensure the system continues to function correctly after release, resulting in proper planning and controlled execution throughout the project.

(c) Explain the importance of SDLC in delivering reliable software.

- Reduces risk and improves quality

Answer: SDLC is important in delivering reliable software because it reduces the risk of errors by catching issues early through structured testing and review at each phase. It also improves the overall quality of the Online Shopping System by ensuring that every requirement is addressed, documented and verified before the system is deployed to customers.

4. Consider a project where requirements are clearly defined from the beginning.

(a) Explain different SDLC models:

- i. Waterfall Model
- ii. Iterative Model
- iii. Spiral Model
- iv. Agile Model

Answer: The Waterfall Model follows a strict sequential process where each phase must be completed before the next begins, making it suitable for projects with fixed requirements. The Iterative Model develops the system through repeated cycles, refining it with each iteration. The Spiral Model combines iterative development with risk analysis at every cycle, making it suitable for large, complex projects. The Agile Model breaks the project into small increments called sprints, allowing continuous feedback and flexibility to change requirements throughout development.

(b) Compare Waterfall and Agile models.

- Waterfall: Sequential, rigid
- Agile: Flexible, iterative

Answer: The Waterfall Model is sequential and rigid, requiring all requirements to be finalized before development starts, which makes it difficult to accommodate changes later. The Agile Model, in contrast, is flexible and iterative, allowing requirements to evolve throughout the project and enabling the team to respond quickly to customer feedback.

(c) Suggest a suitable model for dynamic requirements and justify.

- Agile model is suitable due to adaptability

Answer: For dynamic and evolving requirements, the Agile Model is the most suitable choice because it develops the system in short iterations or sprints, allowing continuous customer feedback and quick adaptation to changing needs. Its flexible structure means that new or changed requirements can be incorporated at any stage without disrupting the entire project, unlike rigid models such as Waterfall.

5. Suppose a system analyst is designing an **ATM System**.

(a) Explain the importance of **Unified Modeling Language (UML)** in OOAD.

- Visual representation of system

Answer: The Unified Modeling Language is important in OOAD because it provides a standard visual representation of the ATM System's structure and behaviour, allowing analysts, designers and developers to communicate the design clearly and consistently before implementation begins.

(b) Describe common UML diagrams:

- i. Use Case Diagram
- ii. Class Diagram
- iii. Sequence Diagram
- iv. Activity Diagram

Answer: A Use Case Diagram shows the interactions between actors, such as the customer and the bank, and the system's functions, such as withdraw cash or check balance. A Class Diagram represents the classes involved in the ATM System, such as Account and Transaction, along with their attributes and relationships. A Sequence Diagram illustrates the order of messages exchanged between objects during a transaction, such as a cash withdrawal. An Activity Diagram models the workflow or steps involved in a process, such as the sequence of actions from card insertion to cash dispensing.

(c) Explain how UML supports object-oriented methodologies.

- Helps in analysis, design, and communication

Answer: UML supports object-oriented methodologies by providing standard notations that assist in the analysis, design and communication of the ATM System throughout its development. It allows developers to visualize the structure and behaviour of the system before coding, making it easier to identify design flaws early and ensuring that all stakeholders share a common understanding of the system.

6. Consider a scenario where a system needs to be designed for **Hospital Management**.

(a) Explain structured design approach in software development.

- Dividing system into modules

Answer: The structured design approach for the Hospital Management System involves dividing the overall system into smaller, manageable modules such as patient registration,

appointment scheduling and billing, each of which performs a specific function and can be developed and tested independently.

(b) Compare structured approach with object-oriented approach.

- Structured: Function-based
- OO: Object-based

Answer: The structured approach is function-based, focusing on breaking the system down into a hierarchy of functions or procedures that operate on data, whereas the object-oriented approach is object-based, organizing the system around objects that combine both data and the behaviour that operates on that data, such as Patient and Doctor objects in the Hospital Management System.

(c) Explain how modular design improves system efficiency and maintenance.

- Easy debugging, scalability, and reuse

Answer: Modular design improves system efficiency and maintenance because each module, such as billing or patient records, can be debugged, updated or replaced without affecting the rest of the Hospital Management System. This makes the system easier to scale as new features are added and allows components to be reused in other parts of the system or in future projects.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps